

CS 224

LAB 4

Preliminary Design Report

Section: 6

Arda Akin

22402316

April 4, 2026

Part (B) - Hazards

- Load-use Hazard

Type: Data hazard

Stages affected: Decode, Execute

Explanation: A lw instruction reads memory in execute stage, the result is not available until

Solution: 1 cycle stall + forwarding

- Load-store Hazard

Type: Data hazard

Stages affected: Memory, Execute

Explanation: A lw is followed by a sw that stores the loaded value. The load result is available 2 stages ahead.

Solution: W Forwarding

- Compute-use Hazard

Type: Data Hazard

Stages affected: Execute, Decode

Explanation: An instruction in execute produces a result that another instruction in decode needs as a source operand.

Solution: EX forwarding

- BEQ Branch Hazard

Type: Control Hazard

Stages affected: Decode, Fetch

Explanation: PC+4 is already fetched before branch is decoded.

Solution: Flush the wrongly fetched instruction, forward if memory is needed.

- J-Type Jump Hazard

Type: Control Hazard

Stages affected: Decode, Fetch

Explanation: PC+4 is already fetched before jump is decoded.

Solution: Flush the wrongly fetched instruction and redirect PC to jump target

- Branch After Load Hazard

Type: Data Hazard

Stages affected: Decode / Execute

Explanation: A lw is immediately followed by beq, beq needs the value in decode but it is not loaded to memory until the end of the memory stage.

Solution: 2-cycle stall

Part (C) - Logic Equations

ForwardAE:

If (RegWriteM & (WriteRegM != 0) & (WriteRegM == rsE))

ForwardAE = 2'b01

Elif (RegWriteW & (WriteRegW != 0) & (WriteRegW == rsE))

ForwardAE = 2'b10

Else

ForwardAE = 2'b00

ForwardBE:

If (RegWriteM & (WriteRegM != 0) & (WriteRegM == rtE))

ForwardBE = 2'b01

Elif (RegWriteW & (WriteRegW != 0) & (WriteRegW == rtE))

ForwardBE = 2'b10

Else

ForwardBE = 2'b00

ForwardAD:

ForwardAD = RegWriteM & (WriteRegM != 0) & (WriteRegM == rsD)

Forward BD:

ForwardBD = RegWriteM & (WriteRegM != 0) & (WriteRegM == rtD)

LwStall:

$lwStall = MemToRegE \ \& \ ((WriteRegE == rsD) \ | \ (WriteRegE == rtD))$

BranchStall:

$branchStall = BranchD \ \& \ RegWriteE \ \& \ ((WriteRegE == rsD) \ | \ (WriteRegE == rtD))$

$StallF = lwStall \ | \ branchStall$

$StallD = lwStall \ | \ branchStall$

$FlushE = lwStall \ | \ branchStall$

Part (D) - System Verilog Code

// Define pipes that exist in the PipelinedDatapath. // The pipe between Writeback (W) and Fetch (F), as well as Fetch (F) and Decode (D) is given to you. // However, you can change them, if you want. // Create the rest of the pipes where inputs follow the naming conventions in the book.

```
module PipeFtoD(input logic[31:0] instr, PcPlus4F,  
                input logic EN, clk, // StallD will be connected as this EN  
                output logic[31:0] instrD, PcPlus4D);  
  
    always_ff @(posedge clk)  
        if(EN)  
            begin  
                instrD<=instr;  
                PcPlus4D<=PcPlus4F;  
            end  
  
endmodule
```

// Similarly, the pipe between Writeback (W) and Fetch (F) is given as follows.

```
module PipeWtoF(input logic[31:0] PC,  
                input logic EN, clk, // StallF will be connected as this EN  
                output logic[31:0] PCF);  
  
    always_ff @(posedge clk)  
        if(EN)  
            begin  
                PCF<=PC;  
            end  
  
endmodule
```

```
endmodule
```

```
// *****
```

```
// Below, write the modules for the pipes PipeDtoE, PipeEtoM, PipeMtoW yourselves.
```

```
// Don't forget to connect Control signals in these pipes as well.
```

```
// *****
```

```
module PipeDtoE( input logic clk, FlushE,  
    input logic RegWriteD, MemToRegD, MemWriteD,  
    input logic ALUSrcD, RegDstD, BranchD, JumpD,  
    input logic [2:0] ALUControlD,  
    input logic [31:0] RD1D, RD2D, SignImmD, PcPlus4D,  
    input logic [4:0] rsD, rtD, rdD,  
    output logic RegWriteE, MemToRegE, MemWriteE,  
    output logic ALUSrcE, RegDstE, BranchE, JumpE,  
    output logic [2:0] ALUControlE,  
    output logic [31:0] RD1E, RD2E, SignImmE, PcPlus4E,  
    output logic [4:0] rsE, rtE, rdE  
);  
    always_ff @(posedge clk) begin  
        if (FlushE) begin  
            RegWriteE <= 0; MemToRegE <= 0; MemWriteE <= 0;  
            ALUSrcE <= 0; RegDstE <= 0; BranchE <= 0; JumpE <= 0;  
            ALUControlE <= 3'b000;  
            RD1E <= 0; RD2E <= 0; SignImmE <= 0; PcPlus4E <= 0;  
            rsE <= 0; rtE <= 0; rdE <= 0;  
        end else begin  
            {RegWriteE,MemToRegE,MemWriteE,ALUSrcE,RegDstE,BranchE,JumpE}  
            <=  
            {RegWriteD,MemToRegD,MemWriteD,ALUSrcD,RegDstD,BranchD,JumpD  
};  
            ALUControlE <= ALUControlD;
```

```

        {RD1E,RD2E,SignImmE,PcPlus4E} <=
        {RD1D,RD2D,SignImmD,PcPlus4D};

        {rsE,rtE,rdE} <= {rsD,rtD,rdD};

    end

end

endmodule

module PipeEtoM( input logic clk,

    input logic RegWriteE, MemToRegE, MemWriteE, BranchE, JumpE,

    input logic ZeroE,

    input logic [31:0] ALUOutE, WriteDataE, PCBranchE,

    input logic [4:0] WriteRegE,

    output logic RegWriteM, MemToRegM, MemWriteM, BranchM, JumpM,

    output logic ZeroM,

    output logic [31:0] ALUOutM, WriteDataM, PCBranchM,

    output logic [4:0] WriteRegM

);

    always_ff @(posedge clk) begin

        {RegWriteM,MemToRegM,MemWriteM,BranchM,JumpM}

        <= {RegWriteE,MemToRegE,MemWriteE,BranchE,JumpE};

        ZeroM <= ZeroE;

        {ALUOutM,WriteDataM,PCBranchM} <= {ALUOutE,WriteDataE,PCBranchE};

        WriteRegM <= WriteRegE;

    end

endmodule

module PipeMtoW( input logic clk,

    input logic RegWriteM, MemToRegM,

    input logic [31:0] ReadDataM, ALUOutM,

```

```

        input logic [4:0] WriteRegM,
        output logic RegWriteW, MemToRegW,
        output logic [31:0] ReadDataW, ALUOutW,
        output logic [4:0] WriteRegW
    );

    always_ff @(posedge clk) begin
        {RegWriteW,MemToRegW} <= {RegWriteM,MemToRegM};
        {ReadDataW,ALUOutW} <= {ReadDataM,ALUOutM};
        WriteRegW <= WriteRegM;
    end

endmodule

//*****
// End of the individual pipe definitions.

//*****

//*****
// Below is the definition of the datapath.

// The signature of the module is given. The datapath will include (not limited to) the
// following items:

// (1) Adder that adds 4 to PC
// (2) Shifter that shifts SignImmE to left by 2
// (3) Sign extender and Register file
// (4) PipeFtoD
// (5) PipeDtoE and ALU
// (5) Adder for PCBranchM
// (6) PipeEtoM and Data Memory
// (7) PipeMtoW
// (8) Many muxes
// (9) Hazard unit
// ...?

//*****

```

```

module datapath (
    input logic    clk, reset,
    input logic    RegWriteD, MemToRegD, MemWriteD,
    input logic    ALUSrcD, RegDstD, BranchD, JumpD,
    input logic [2:0] ALUControlD,
    output logic    MemWriteM,
    output logic [31:0] ALUOutM, WriteDataM,
    output logic [31:0] PCF,
    output logic [31:0] instrF,
    output logic [31:0] ResultW
);

    // Hazard unit wires
    logic    StallF, StallD, FlushE, FlushD;
    logic [1:0] ForwardAE, ForwardBE;
    logic    ForwardAD, ForwardBD;

    // Fetch stage
    logic [31:0] PC_next;
    logic [31:0] PcPlus4F;

    // Decode stage
    logic [31:0] instrD, PcPlus4D;
    logic [31:0] RD1D, RD2D;
    logic [31:0] RD1D_fwd, RD2D_fwd;
    logic [31:0] SignImmD, ShiftedImmD, PCBranchD;
    logic    EqualD, PcSrcD;
    logic [4:0] rsD, rtD, rdD;

    // Execute stage

```

```

        logic      RegWriteE, MemToRegE, MemWriteE;

        logic      ALUSrcE, RegDstE, BranchE, JumpE;

        logic [2:0] ALUControlE;

        logic [31:0] RD1E, RD2E, SignImmE, PcPlus4E;

        logic [4:0] rsE, rtE, rdE;

        logic [31:0] SrcAE, SrcBE, WriteDataE;

        logic [31:0] ALUOutE, ShiftedImmE, PCBranchE;

        logic      ZeroE;

        logic [4:0] WriteRegE;


        // Memory stage

        logic      RegWriteM_s, MemToRegM, MemWriteM_s, BranchM,
JumpM;

        logic      ZeroM;

        logic [31:0] ALUOutM_s, WriteDataM_s, PCBranchM;

        logic [4:0] WriteRegM;

        logic [31:0] ReadDataM;

        logic      PcSrcM;


        // Writeback stage

        logic      RegWriteW, MemToRegW;

        logic [31:0] ReadDataW, ALUOutW;

        logic [4:0] WriteRegW;


        // FETCH STAGE

        // PC mux: Jump > Branch > sequential

        assign PcSrcM = BranchM & ZeroM;


        always_comb begin

```

```

        if (JumpM) PC_next = {PCBranchM[31:28], instrD[25:0], 2'b00};
        else if (PcSrcM) PC_next = PCBranchM;
        else          PC_next = PcPlus4F;
    end

    // PC register with stall support
    always_ff @(posedge clk or posedge reset) begin
        if (reset)      PCF <= 32'h0;
        else if (~StallF) PCF <= PC_next;
    end

    assign PcPlus4F = PCF + 4;
    imem im1(PCF[7:2], instrF);
    // F -> D PIPE (stall via EN, flush via FlushD)
    always_ff @(posedge clk or posedge reset) begin
        if (reset || FlushD) begin
            instrD <= 32'h0;
            PcPlus4D <= 32'h0;
        end else if (~StallD) begin
            instrD <= instrF;
            PcPlus4D <= PcPlus4F;
        end
    end

    end

    // DECODE STAGE
    assign rsD = instrD[25:21];
    assign rtD = instrD[20:16];
    assign rdD = instrD[15:11];

    regfile rf(clk, RegWriteW, rsD, rtD, WriteRegW, ResultW, RD1D, RD2D);

```

```

signext se(instrD[15:0], SignImmD);

sl2    sl_D(SignImmD, ShiftedImmD);

adder  branch_adder_D(PcPlus4D, ShiftedImmD, PCBranchD);


// Forwarding muxes for early branch comparator

mux2 #(32) fwdAD_mux(RD1D, ALUOutM_s, ForwardAD, RD1D_fwd);
mux2 #(32) fwdBD_mux(RD2D, ALUOutM_s, ForwardBD, RD2D_fwd);


assign EqualD = (RD1D_fwd == RD2D_fwd);
assign PcSrcD = BranchD & EqualD;    // drives FlushD in hazard unit


// D -> E PIPE

PipeDtoE pDtoE(
    clk, FlushE,
    RegWriteD, MemToRegD, MemWriteD,
    ALUSrcD, RegDstD, BranchD, JumpD,
    ALUControlD,
    RD1D, RD2D, SignImmD, PcPlus4D,
    rsD, rtD, rdD,
    RegWriteE, MemToRegE, MemWriteE,
    ALUSrcE, RegDstE, BranchE, JumpE,
    ALUControlE,
    RD1E, RD2E, SignImmE, PcPlus4E,
    rsE, rtE, rdE
);

// EXECUTE STAGE

// WriteReg: rd (R-type) or rt (I-type)
mux2 #(5) wreg_mux(rtE, rdE, RegDstE, WriteRegE);

```

```
// 3-way forwarding mux for ALU source A
```

```
always_comb begin
```

```
    case (ForwardAE)
```

```
        2'b01: SrcAE = ALUOutM_s;
```

```
        2'b10: SrcAE = ResultW;
```

```
        default: SrcAE = RD1E;
```

```
    endcase
```

```
end
```

```
// 3-way forwarding mux for ALU source B (before ALUSrc mux)
```

```
always_comb begin
```

```
    case (ForwardBE)
```

```
        2'b01: WriteDataE = ALUOutM_s;
```

```
        2'b10: WriteDataE = ResultW;
```

```
        default: WriteDataE = RD2E;
```

```
    endcase
```

```
end
```

```
// ALUSrc mux: forwarded RD2 vs sign-extended immediate
```

```
mux2 #(32) alusrc_mux(WriteDataE, SignImmE, ALUSrcE, SrcBE);
```

```
alu alu1(SrcAE, SrcBE, ALUControlE, ALUOutE, ZeroE);
```

```
// Branch target (carried to M stage, used if branch resolves there)
```

```
sl2 sl_E(SignImmE, ShiftedImmE);
```

```
adder branch_adder_E(PcPlus4E, ShiftedImmE, PCBranchE);
```

```
// E -> M PIPE
```

```

PipeEtoM pEtoM(
    clk,
    RegWriteE, MemToRegE, MemWriteE, BranchE, JumpE,
    ZeroE,
    ALUOutE, WriteDataE, PCBranchE,
    WriteRegE,
    RegWriteM_s, MemToRegM, MemWriteM_s, BranchM, JumpM,
    ZeroM,
    ALUOutM_s, WriteDataM_s, PCBranchM,
    WriteRegM
);

// MEMORY STAGE
dmem dm1(clk, MemWriteM_s, ALUOutM_s, WriteDataM_s,
ReadDataM);

```

```

assign MemWriteM = MemWriteM_s;
assign ALUOutM = ALUOutM_s;
assign WriteDataM = WriteDataM_s;

```

```

// M -> W PIPE

```

```

PipeMtoW pMtoW(
    clk,
    RegWriteM_s, MemToRegM,
    ReadDataM, ALUOutM_s,
    WriteRegM,
    RegWriteW, MemToRegW,
    ReadDataW, ALUOutW,
    WriteRegW

```

```
);
```

```
// WRITEBACK STAGE
```

```
mux2 #(32) result_mux(ALUOutW, ReadDataW, MemToRegW, ResultW);
```

```
// HAZARD UNIT
```

```
HazardUnit hu(
```

```
    RegWriteW, WriteRegW,
```

```
    RegWriteM_s, MemToRegM, WriteRegM,
```

```
    RegWriteE, MemToRegE, WriteRegE,
```

```
    rsE, rtE,
```

```
    rsD, rtD,
```

```
    PcSrcD,
```

```
    ForwardAE, ForwardBE,
```

```
    ForwardAD, ForwardBD,
```

```
    FlushE, FlushD, StallD, StallF
```

```
);
```

```
endmodule
```

```
module HazardUnit( input logic RegWriteW,  
    input logic [4:0] WriteRegW,  
    input logic RegWriteM,MemToRegM,  
    input logic [4:0] WriteRegM,  
    input logic RegWriteE,MemToRegE,  
    input logic [4:0] rsE,rtE,  
    input logic [4:0] rsD,rtD,  
    output logic [2:0] ForwardAE,ForwardBE,
```

```

        output logic FlushE, StallD, StallF
    );

    always_comb begin
        // EX forwarding
        if (RegWriteM && WriteRegM!=0 && WriteRegM==rsE) ForwardAE = 2'b01;
        else if (RegWriteW && WriteRegW!=0 && WriteRegW==rsE) ForwardAE = 2'b10;
        else ForwardAE = 2'b00;

        if (RegWriteM && WriteRegM!=0 && WriteRegM==rtE) ForwardBE = 2'b01;
        else if (RegWriteW && WriteRegW!=0 && WriteRegW==rtE) ForwardBE = 2'b10;
        else ForwardBE = 2'b00;

        // Decode forwarding for early branch
        ForwardAD = RegWriteM & (WriteRegM!=0) & (WriteRegM==rsD);
        ForwardBD = RegWriteM & (WriteRegM!=0) & (WriteRegM==rtD);

        // Load-use stall lwStall = MemToRegE & ((WriteRegE==rsD)|(WriteRegE==rtD));
        StallF = lwStall;
        StallD = lwStall;

        FlushE = lwStall | (BranchD & RegWriteE &
        ((WriteRegE==rsD)|(WriteRegE==rtD)));
    end

endmodule

module mips (input logic clk, reset,
    output logic[31:0] pc,
    input logic[31:0] instr,
    output logic memwrite,
    output logic[31:0] aluout, resultW,
    output logic[31:0] instrOut,
    input logic[31:0] readdata);

    logic memtoreg, pcsrc, zero, alusrc, regdst, regwrite, jump;
    logic [2:0] alucontrol;

```

```
assign instrOut = instr;
```

```
    controller ctrl(
        instrF[31:26], instrF[5:0],
        memtoreg, memwrite_s, alusrc,
        regdst, regwrite, jump,
        alucontrol, branch );

    datapath dp( clk, reset,
        regwrite, memtoreg, memwrite_s,
        alusrc, regdst, branch, jump,
        alucontrol, memwrite, aluout,
        resultW, pc, instrF, resultW
    );
```

```
endmodule
```

```
module imem ( input logic [5:0] addr, output logic [31:0] instr);
```

```
// imem is modeled as a lookup table, a stored-program byte-addressable ROM always_comb
case ({addr,2'b00}) // word-aligned fetch // //
```

```
*****
```

```
// Here, you can paste your own test cases that you prepared for the part 1-g. // Below is a
program from the single-cycle lab. //
```

```
*****
```

```
// // address instruction // ----- 8'h00: instr = 32'h20020005; // disassemble, by
hand 8'h04: instr = 32'h2003000c; // or with a program, 8'h08: instr = 32'h2067fff7; // to find
out what 8'h0c: instr = 32'h00e22025; // this program does! 8'h10: instr = 32'h00642824;
8'h14: instr = 32'h00a42820; 8'h18: instr = 32'h10a7000a; 8'h1c: instr = 32'h0064202a; 8'h20:
instr = 32'h10800001; 8'h24: instr = 32'h20050000; 8'h28: instr = 32'h00e2202a; 8'h2c: instr
= 32'h00853820; 8'h30: instr = 32'h00e23822; 8'h34: instr = 32'hac670044; 8'h38: instr =
32'h8c020050; 8'h3c: instr = 32'h08000011; 8'h40: instr = 32'h20020001; 8'h44: instr =
32'hac020054; 8'h48: instr = 32'h08000012; // j 48, so it will loop here default: instr =
{32{1'bx}}; // unknown address endcase endmodule
```

```
//
```

```
*****
```

```
// Below are the modules that you shouldn't need to modify at all. //
```

```
*****
```

```
module controller(input logic[5:0] op, funct,
```

```

        output logic memtoreg, memwrite,
        output logic alusrc, output logic regdst, regwrite,
        output logic jump,
        output logic[2:0] alucontrol, output logic branch);
logic [1:0] aluop;
maindec md (op, memtoreg, memwrite, branch, alusrc, regdst, regwrite, jump, aluop);
aludec ad (funct, aluop, alucontrol);
endmodule

// External data memory used by MIPS single-cycle processor
module dmem (input logic clk, we,
             input logic[31:0] a, wd,
             output logic[31:0] rd);
logic [31:0] RAM[63:0];
assign rd = RAM[a[31:2]]; // word-aligned read (for lw)
always_ff @(posedge clk)
if (we)
RAM[a[31:2]] <= wd; // word-aligned write (for sw)
endmodule

module maindec (input logic[5:0] op,
               output logic memtoreg, memwrite, branch,
               output logic alusrc, regdst, regwrite, jump,
               output logic[1:0] aluop );
logic [8:0] controls;
assign {regwrite, regdst, alusrc, branch, memwrite, memtoreg, aluop, jump} = controls;
always_comb
case(op)
6'b000000: controls <= 9'b110000100; // R-type
6'b100011: controls <= 9'b101001000; // LW

```

```

6'b101011: controls <= 9'b001010000; // SW
6'b000100: controls <= 9'b000100010; // BEQ
6'b001000: controls <= 9'b101000000; // ADDI
6'b000010: controls <= 9'b000000001; // J
default: controls <= 9'bxxxxxxxx; // illegal op
endcase

endmodule

module aludec (input logic[5:0] funct,
               input logic[1:0] aluop,
               output logic[2:0] alucontrol);

always_comb
case(aluop)
2'b00: alucontrol = 3'b010; // add (for lw/sw/addi)
2'b01: alucontrol = 3'b110; // sub (for beq)
default: case(funct) // R-TYPE instructions
        6'b100000: alucontrol = 3'b010; // ADD
        6'b100010: alucontrol = 3'b110; // SUB
        6'b100100: alucontrol = 3'b000; // AND
        6'b100101: alucontrol = 3'b001; // OR
        6'b101010: alucontrol = 3'b111; // SLT
        default: alucontrol = 3'bxxx; // ???
        endcase
endcase

endmodule

module regfile (input logic clk, we3,
               input logic[4:0] ra1, ra2, wa3,
               input logic[31:0] wd3,
               output logic[31:0] rd1, rd2);

```

```

logic [31:0] rf [31:0];

// three ported register file: read two ports combinationaly
// write third port on rising edge of clock. Register0 hardwired to 0.
always_ff @(negedge clk)
if (we3)
    rf [wa3] <= wd3;
assign rd1 = (ra1 != 0) ? rf [ra1] : 0;
assign rd2 = (ra2 != 0) ? rf [ra2] : 0;
endmodule

module alu(input logic [31:0] a, b,
    input logic [2:0] alucont,
    output logic [31:0] result,
    output logic zero);

always_comb
case(alucont)
    3'b010: result = a + b;
    3'b110: result = a - b;
    3'b000: result = a & b;
    3'b001: result = a | b;
    3'b111: result = (a < b) ? 1 : 0;
    default: result = {32{1'bx}};
endcase

assign zero = (result == 0) ? 1'b1 : 1'b0;
endmodule

module adder (input logic[31:0] a, b,
    output logic[31:0] y);

assign y = a + b;

endmodule

module sl2 (input logic[31:0] a,

```

```

        output logic[31:0] y);
assign y = {a[29:0], 2'b00}; // shifts left by 2

endmodule

module signext (input logic[15:0] a,
                output logic[31:0] y);
assign y = {{16{a[15]}}, a}; // sign-extends 16-bit a
endmodule

// parameterized register
module flopr #(parameter WIDTH = 8)
    (input logic clk, reset,
     input logic[WIDTH-1:0] d,
     output logic[WIDTH-1:0] q);
always_ff@(posedge clk, posedge reset)
    if (reset) q <= 0;
    else q <= d;
endmodule

// parameterized 2-to-1 MUX
module mux2 #(parameter WIDTH = 8)
    (input logic[WIDTH-1:0] d0, d1,
     input logic s,
     output logic[WIDTH-1:0] y);
assign y = s ? d1 : d0;
endmodule

```

Part (E) - Test Programs

- Test Program 1 – No Hazards

```
addi $t0, $0, 5 # 0x00          0x20080005    # $t0 = 5
```

addi \$t1, \$0, 3 # 0x04	0x20090003	# \$t1 = 3
addi \$t2, \$0, 10 # 0x08	0x200a000a	# \$t2 = 10
nop	# 0x0c	0x00000000
nop	# 0x10	00000000
add \$t3, \$t0, \$t1 # 0x14	01095820	# \$t3 = 8 (no hazard — 2 NOPs)
nop	# 0x18	00000000

Expected result: \$t3 = 0x00000008. No stalls. No forwarding needed.

- Test Program 2 – EX (Compute-Use) Forwarding Hazard

addi \$s0, \$0, 7 # 0x1c	20100007	# \$s0 = 7
add \$s1, \$s0, \$s0 # 0x20	02108820	# \$s1 = 14

Hazard: add uses \$s0 which is produced by addi still in E/M.

ForwardAE = ForwardBE = 2'b01 (forward from M stage ALUOutM = 7).

Expected result: \$s1 = 0x0000000E = 14.

- Test Program 3 – Load Use Stall

sw \$zero, 0x50(\$0) # (pre-load store to set MEM[80] = 0 for test, done externally)

lw \$s0, 0x50(\$0) # 0x24	8c100050	# \$s0 = MEM[80]
add \$s1, \$s0, \$s0 # 0x28	02108820	# \$s1 = 2*MEM[80]

Hazard: MemToRegE=1 and WriteRegE=\$s0 matches rsD=\$s0 and rtD=\$s0.

HazardUnit asserts StallF=StallD=1 and FlushE=1 for one cycle.

Expected Output: \$s1 = 2 * MEM[80]. Simulation shows one extra NOP cycle inserted

- Test Program 4 – Branch Hazard

addi \$t0, \$0, 5 # 0x2c	20080005
addi \$t1, \$0, 5 # 0x30	20090005
nop	# 0x34 00000000
nop	# 0x38 00000000

beq \$t0, \$t1, +1	# 0x3c	11090001	# branch taken -> PC = 0x44
addi \$t2, \$0, 99	# 0x40	20120063	# MUST be flushed
addi \$t3, \$0, 42	# 0x44	2013002a	# branch target: \$t3 = 42
j 0x48	# 0x48	08000012	# halt loop

Hazard: BEQ resolved in Decode. When taken, PcSrcD=1 resets PipeFtoD, flushing the instruction at 0x40.

Expected Output: \$t3 = 0x0000002A = 42. \$t2 remains 0 (flushed).